AI-Based Pre-Placement Trainer

Bug Fix Report & Enhancement Blueprint

Complete Technical Improvement Guide — Version 2.0

Project	AI Pre-Placement Trainer & Feedback Model
Team	Vedant Kumbhar, Meeraj Krishna M.R., Loukik Ingale
Guide	Dr. Suraj R. Nalawade
Document Date	March 2026
Total Issues Identified	**6 Bugs + 4 Major Enhancements**

1. Executive Summary

This document provides a complete technical analysis of all identified bugs and required enhancements for the AI-Based Pre-Placement Trainer project. Each issue has been studied in detail, and a concrete, step-by-step solution has been provided. The goal of this document is to serve as a developer implementation guide so that the project can be upgraded from its current state to a production-ready, industry-level platform.

The issues identified fall into three categories:

- Frontend Validation Bugs — problems that affect data quality at the input stage.
- Backend / Logic Bugs — problems where the system produces incorrect or repetitive results.
- Major Feature Gaps — areas where the existing system falls significantly short of industry expectations.

Issue / Feature	Priority	Fix / Solution
BUG-01: Name field accepts invalid characters	Critical	Add regex validation on frontend and backend
BUG-02: Aptitude learning content is too thin	High	Restructure content with rich HTML + expand database
BUG-03: Quiz always shows same 5 questions	Critical	Implement randomized question selection in backend
BUG-04: Interview evaluates one answer at a time	High	Redesign to batch-submit all answers at session end
BUG-05: STT (Speech-to-Text) does not work reliably	Critical	Replace with Web Speech API + silence detection
FEAT-01: Real-time interview experience (TTS + STT flow)	High	Full redesign of interview UI with TTS, silence detection
FEAT-02: Auto-save answers to browser localStorage	Medium	Save Q&A as JSON keyed by question ID in localStorage
FEAT-03: Send all answers at once for unbiased AI feedback	High	Single POST with all answers at session end
FEAT-04: Downloadable PDF report of interview results	Medium	Generate PDF on frontend using jsPDF after feedback

2. Existing Codebase Drawbacks Analysis

Before listing individual fixes, this section documents the structural weaknesses observed in the current project architecture that contribute to multiple bugs at once.

2.1 Frontend — React

- No centralized form validation library is used. Each form component validates independently and inconsistently, leading to different validation behaviors across the app.
- The interview component triggers an API call immediately after each answer is submitted, creating a waterfall of sequential API requests. This makes the interview feel slow and choppy instead of fluid.
- There is no local state persistence (localStorage/sessionStorage) for interview sessions, so if the tab is accidentally closed, all progress is lost.
- The STT integration uses a custom wrapper around the Web Speech API but fails to handle browser compatibility, silence detection, or interim results correctly.
- No TTS (Text-to-Speech) is implemented for reading questions aloud, making the experience purely text-based and unlike a real interview.
- The PDF generation for reports is entirely missing.

2.2 Backend — Flask

- The question selection query for the aptitude quiz uses a simple LIMIT 5 without ORDER BY RANDOM(), causing the same questions to always appear first in the database result set.
- The /submit-answer endpoint processes and evaluates one answer at a time by calling the OpenAI API per answer. This is expensive in terms of API tokens and causes visible delays after every answer.
- Name validation is done only in the database (unique constraint) but not at the application logic layer. There is no check on character type or format.
- Resume parsing is limited: if spaCy cannot find a skill entity, the technical questions fall back to generic questions unrelated to the resume, making the system appear broken.
- Error messages returned from the API are technical Python exceptions rather than user-friendly messages.

2.3 Content — Aptitude Module

- The learning content for each topic is stored as a short HTML string in the database. This severely limits the depth, formatting, and richness of the educational material.
- There is no video embedding support in the current UI component, so video_url fields in the database are stored but never displayed.
- The question bank has insufficient variety. With only a handful of questions per topic, repetition is guaranteed within a few practice sessions.
- No difficulty levels are implemented in question selection, so all users (beginner to advanced) receive the same questions.

3. Bug Fix — BUG-01: Name Field Validation

Bug Description

Location: Registration page (Frontend) + /api/auth/register endpoint (Backend)

Symptom: Users can type numbers, special characters, and symbols in the 'Full Name' field during registration.

Impact: Corrupted user records in the database; names like '123@#$' make the system look unprofessional and break dashboard display logic.

3.1 Root Cause

The registration form in the React component has no onChange validation for the name field. The input is sent directly to the Flask backend, which only checks for uniqueness before inserting. Neither side checks whether the name contains only valid name characters.

3.2 Fix — Frontend (React)

Add a regex validator directly on the name input field. Use the onChange event to test the input against the pattern and set an error state.

Step 1 — Add state for error message in your Registration component

```
const [nameError, setNameError] = useState('');
```

Step 2 — Create the validation function

```
const validateName = (value) => {
  const nameRegex = /^[a-zA-Z\s'-]{2,50}$/;
  // Allows: letters (upper and lower), spaces, hyphen, apostrophe
  // Min 2 chars, Max 50 chars
  if (!value.trim()) return 'Name is required.';
  if (!nameRegex.test(value)) return 'Name can only contain letters, spaces, hyphens, and
apostrophes.';
  return '';
};
```

Step 3 — Attach to the input field

```
<input
  type='text'
  value={name}
  onChange={(e) => {
```

```
    const val = e.target.value;
    // Only allow the regex characters as the user types
    if (/^[a-zA-Z\s'-]*$/.test(val) || val === '') {
      setName(val);
      setNameError(validateName(val));
    }
    // If invalid character typed, do nothing (character never appears)
  }}
  onBlur={() => setNameError(validateName(name))}
/>
{nameError && <span className='error-text'>{nameError}</span>}
```

3.3 Fix — Backend (Flask)

Even with frontend validation, always validate on the backend as a safety net. Add this check in your register route before any database operation:

Add to /api/auth/register route in Flask

```
import re

@app.route('/api/auth/register', methods=['POST'])
def register():
    data = request.get_json()
    name = data.get('name', '').strip()

    # Validate name
    name_pattern = re.compile(r"^[a-zA-Z\s'\-]{2,50}$")
    if not name_pattern.match(name):
        return jsonify({'error': 'Name must contain only letters (2-50 characters).'}), 400

    # ... rest of your registration logic
```

3.4 Additional Input Validations to Add

- Email: Use HTML type='email' plus backend regex: r'^[\w.-]+@[\w.-]+\.\w{2,}$'
- Password: Minimum 8 chars, at least one uppercase, one number, one special character.
- Username: Only alphanumerics and underscore, 3-20 chars: r'^[a-zA-Z0-9_]{3,20}$'

4. Bug Fix — BUG-02: Thin Aptitude Learning Content

Bug Description

Location: Topics table in database + Topic detail page in React

Symptom: The learning content for each topic shows only a few lines of text with no structure, examples, visuals, or depth.

Impact: Students cannot learn from the platform and skip to quizzes without understanding concepts — defeating the learning purpose.

4.1 Root Cause

The learning_html column in the Topics table stores short plain text. The React component renders it via dangerouslySetInnerHTML without any structured styling. No video integration, no examples, no practice problems within the content itself.

4.2 Solution Strategy — Three-Part Approach

Part A: Restructure the Topic Schema

Expand the Topic model in Flask/SQLAlchemy to support richer content:

New Topic model fields

```
class Topic(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), unique=True, nullable=False)
    description = db.Column(db.String(500))
    learning_html = db.Column(db.Text)       # Full rich HTML content
    video_url = db.Column(db.String(500))    # YouTube embed URL
    video_title = db.Column(db.String(200))  # Video title for display
    reference_links = db.Column(db.Text)     # JSON array of {title, url}
    key_formulas = db.Column(db.Text)        # JSON array of formula strings
    difficulty = db.Column(db.String(20))    # 'Beginner', 'Intermediate', 'Advanced'
    estimated_read_minutes = db.Column(db.Integer)
```

Part B: Write Rich HTML Content for Each Topic

For each topic, replace the short text with well-structured HTML. Below is the template to follow for EVERY topic. Fill in the actual content specific to each topic.

Standard Rich HTML Template (copy and fill for each topic)

```
<div class='topic-content'>
  <h2>What is [Topic Name]?</h2>
  <p>Clear definition in 3-4 sentences...</p>

  <h3>Key Concepts</h3>
  <ul>
    <li><strong>Concept 1:</strong> Explanation</li>
    <li><strong>Concept 2:</strong> Explanation</li>
  </ul>

  <h3>Important Formulas</h3>
  <div class='formula-box'>
    Formula = (value / total) x 100
  </div>

  <h3>Solved Example</h3>
  <div class='example-box'>
    <p><strong>Q:</strong> If A can do a job in 10 days...</p>
    <p><strong>Solution:</strong> Step-by-step working...</p>
    <p><strong>Answer:</strong> X days</p>
  </div>

  <h3>Tips & Tricks</h3>
  <ul class='tips-list'>
    <li>Tip 1 for quick solving</li>
  </ul>
</div>
```

Part C: Add Content for All 10 Core Topics

Below is the list of topics that MUST have full content written. Use GeeksForGeeks, IndiaBIX, and PrepInsta as reference sources to write the learning_html for each:

#	Topic	Must Cover	Practice Q Count
1	**Number Systems**	Conversion, Divisibility, HCF/LCM	Minimum 40 questions
2	**Percentages**	Basic %, profit/loss, discount	Minimum 35 questions
3	**Time & Work**	Work rates, pipes & cisterns	Minimum 35 questions
4	**Time, Speed & Distance**	Trains, boats, circular tracks	Minimum 40 questions
5	**Algebra**	Linear equations, quadratic	Minimum 30 questions
6	**Logical Reasoning**	Pattern, series, syllogism	Minimum 50 questions
7	**Data Interpretation**	Bar, pie, line charts	Minimum 30 questions

8	**Verbal Ability**	RC, grammar, vocabulary	Minimum 40 questions
9	**Coding & Decoding**	Letter/number coding	Minimum 30 questions
10	**Probability & Stats**	Basic probability, averages	Minimum 30 questions

Part D: Update the React Topic Page

In your React TopicDetail component, add video embed support and reference links:

Add to your TopicDetail.jsx component

```
// Show embedded YouTube video if available
{topic.video_url && (
  <div className='video-section'>
    <h3>Video Explanation</h3>
    <iframe
      width='100%' height='400'
      src={topic.video_url}   // Use YouTube embed URL format:
      // https://www.youtube.com/embed/VIDEO_ID
      title={topic.video_title}
      frameBorder='0'
      allow='accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope'
      allowFullScreen
    />
  </div>
)}

// Show reference links
{topic.reference_links && JSON.parse(topic.reference_links).map(link => (
  <a key={link.url} href={link.url} target='_blank' rel='noopener noreferrer'>
    {link.title}
  </a>
))}
```

5. Bug Fix — BUG-03: Quiz Always Shows Same Questions

Bug Description

Location: GET /api/aptitude/questions endpoint in Flask

Symptom: Every time a user starts a quiz for a topic, they receive the exact same 5 questions.

Impact: Students memorize answers after 1-2 attempts. No actual learning or practice occurs. The quiz becomes useless.

5.1 Root Cause

The database query fetching questions uses a simple LIMIT without ORDER BY RANDOM(). SQLite (and PostgreSQL) always return rows in the same physical order unless you explicitly ask for random ordering. The query likely looks like this:

BUGGY query (current)

```
questions = Question.query.filter_by(topic_id=topic_id).limit(10).all()
# This always returns the first 10 rows in insertion order
```

5.2 Fix — Backend

Use SQL's random ordering function. In SQLAlchemy with SQLite/PostgreSQL:

FIXED query — simple randomization

```
from sqlalchemy import func

@app.route('/api/aptitude/questions', methods=['GET'])
@jwt_required()
def get_aptitude_questions():
    topic_id = request.args.get('topic_id', type=int)
    difficulty = request.args.get('difficulty', 'all')  # optional filter
    num_questions = request.args.get('count', 10, type=int)  # default 10

    query = Question.query.filter_by(topic_id=topic_id)

    # Filter by difficulty if requested
    if difficulty != 'all':
        query = query.filter_by(difficulty=difficulty)

    # THE KEY FIX: ORDER BY RANDOM()
    questions = query.order_by(func.random()).limit(num_questions).all()
```

```
    if not questions:
        return jsonify({'error': 'No questions found for this topic'}), 404

    return jsonify([q.to_dict() for q in questions])
```

5.3 Additional Improvement — Prevent Repeat in Same Session

To ensure a user never sees the same question twice across multiple sessions on the same day, track which question IDs have been shown:

Add a UserSeenQuestion table

```
class UserSeenQuestion(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    question_id = db.Column(db.Integer, db.ForeignKey('question.id'), nullable=False)
    seen_at = db.Column(db.DateTime, default=datetime.utcnow)

# In the question fetch endpoint, exclude recently seen questions:
recently_seen_ids = [
    r.question_id for r in UserSeenQuestion.query
    .filter_by(user_id=current_user_id)
    .filter(UserSeenQuestion.seen_at > datetime.utcnow() - timedelta(days=1))
    .all()
]
query = query.filter(~Question.id.in_(recently_seen_ids))
```

5.4 Increase the Question Bank

Even with randomization, a small question bank will cycle quickly. Target these minimums per topic for effective practice:

- Quantitative Aptitude topics: minimum 40 questions each.
- Logical Reasoning topics: minimum 50 questions each.
- Verbal Ability topics: minimum 30 questions each.

Use a seed script to bulk-insert questions from a JSON file rather than entering them one by one through the admin panel.

6. Bug Fix — BUG-04 & FEAT-01/02/03: Complete Interview Module Redesign

This is the most significant improvement in the document. The current per-answer evaluation approach has multiple problems that are all solved together with one architectural redesign.

Combined Problem Statement

BUG-04: The interview module calls the AI API after EACH answer. This creates a 10-30 second wait after every question, making the experience feel broken.

BUG-05: STT does not reliably detect silence or browser variations.

FEAT-01: There is no TTS to read questions aloud, so it does not feel like a real interview.

FEAT-02: No localStorage persistence — session lost if tab is closed.

FEAT-03: Per-answer evaluation is biased (evaluated in isolation). All answers should be evaluated together at the end for a holistic, unbiased score.

6.1 New Interview Flow Architecture

The redesigned interview module works as follows — this is the exact flow to implement:

1. **User uploads resume → Flask parses it → Generates 8 questions → Returns question list → Stored in React state AND localStorage.**
2. **Interview page loads. The AI asks Question 1 using browser TTS (text-to-speech). The question text is spoken aloud.**
3. **After TTS finishes speaking, a 2-second countdown timer starts. The user sees a 'Recording starts in 2s...' indicator.**
4. **After 2 seconds, the browser microphone automatically activates. The STT (Web Speech API) starts listening.**
5. **As the user speaks, their transcribed answer appears live on screen (interim results from Web Speech API).**
6. **When the user pauses for 3 continuous seconds of silence, the answer is automatically finalized.**
7. **The finalized answer is saved to localStorage as: key = 'interview_q_{questionId}', value = JSON with question text and answer text.**
8. **The next question is automatically loaded. TTS reads it aloud. Steps 3-7 repeat.**
9. **After all 8 questions are answered, a 'Submit Interview' button appears. User clicks it.**
10. **All 8 Q&A pairs are retrieved from localStorage, packaged into ONE JSON payload, and sent to Flask as a single POST request.**
11. **Flask sends the full conversation to OpenAI API with a single prompt asking for holistic evaluation.**
12. **AI returns scores + feedback for all 8 answers at once. Response is shown on the Results page.**
13. **User can download the complete report as a PDF using jsPDF.**

6.2 Frontend Implementation — React Interview Component

6.2.1 TTS — Reading Questions Aloud

Use the browser's built-in SpeechSynthesis API

```
const speakQuestion = (text) => {
  return new Promise((resolve) => {
    // Cancel any ongoing speech first
    window.speechSynthesis.cancel();

    const utterance = new SpeechSynthesisUtterance(text);
    utterance.rate = 0.9;   // Slightly slower than default
    utterance.pitch = 1.0;
    utterance.volume = 1.0;

    // Choose a natural voice if available
    const voices = window.speechSynthesis.getVoices();
    const preferred = voices.find(v => v.name.includes('Google') || v.name.includes('Natural'));
    if (preferred) utterance.voice = preferred;

    utterance.onend = () => resolve();   // Resolves when speech finishes
    utterance.onerror = () => resolve();  // Also resolve on error to prevent hang

    window.speechSynthesis.speak(utterance);
  });
};
```

6.2.2 STT — Recording with Silence Detection

Robust STT with 3-second silence detection

```
// State variables needed:
const [isRecording, setIsRecording] = useState(false);
const [transcript, setTranscript] = useState('');
const recognitionRef = useRef(null);
const silenceTimerRef = useRef(null);
const SILENCE_THRESHOLD_MS = 3000; // 3 seconds of silence

const startRecording = () => {
  // Browser compatibility check
  const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
  if (!SpeechRecognition) {
    alert('Your browser does not support speech recognition. Please use Google Chrome.');
    return;
```

```
  }

  const recognition = new SpeechRecognition();
  recognition.lang = 'en-IN';  // Indian English accent
  recognition.continuous = true;
  recognition.interimResults = true;

  recognition.onresult = (event) => {
    let finalText = '';
    for (let i = event.resultIndex; i < event.results.length; i++) {
      if (event.results[i].isFinal) finalText += event.results[i][0].transcript + ' ';
    }
    if (finalText) {
      setTranscript(prev => prev + finalText);
      // Reset silence timer whenever new speech detected
      clearTimeout(silenceTimerRef.current);
      silenceTimerRef.current = setTimeout(() => {
        stopRecordingAndAdvance(); // Auto-stop after 3s silence
      }, SILENCE_THRESHOLD_MS);
    }
  };

  recognition.onerror = (event) => {
    if (event.error !== 'no-speech') console.error('STT Error:', event.error);
  };

  recognitionRef.current = recognition;
  recognition.start();
  setIsRecording(true);
  // Start initial silence timer
  silenceTimerRef.current = setTimeout(stopRecordingAndAdvance,
SILENCE_THRESHOLD_MS);
};
```

6.2.3 Main Interview Loop — Orchestrating TTS + Countdown + STT

The central useEffect that drives the full interview flow

```
const runQuestion = async (index) => {
  if (index >= questions.length) {
    setInterviewComplete(true);
    return;
  }

  const question = questions[index];
```

```
  setCurrentQuestion(question);
  setTranscript('');
  setPhase('speaking'); // Show 'AI is speaking...' UI

  // Step 1: TTS reads the question
  await speakQuestion(question.question_text);

  // Step 2: 2-second countdown
  setPhase('countdown');
  await new Promise(resolve => setTimeout(resolve, 2000));

  // Step 3: Auto-start recording
  setPhase('recording');
  startRecording();
};

// When silence detected, stop and go to next question
const stopRecordingAndAdvance = () => {
  recognitionRef.current?.stop();
  clearTimeout(silenceTimerRef.current);
  setIsRecording(false);

  // Save to localStorage
  const answer = transcript || '[No answer given]';
  const key = `interview_q_${currentQuestion.id}`;
  localStorage.setItem(key, JSON.stringify({
    questionId: currentQuestion.id,
    questionText: currentQuestion.question_text,
    questionType: currentQuestion.question_type,
    answerText: answer,
    timestamp: new Date().toISOString()
  }));

  // Move to next question
  setCurrentIndex(prev => prev + 1);
};
```

6.2.4 Collect All Answers and Submit at Once

Gather from localStorage and POST to Flask

```
const submitInterview = async () => {
  const allAnswers = questions.map(q => {
    const stored = localStorage.getItem(`interview_q_${q.id}`);
```

```
    return stored ? JSON.parse(stored) : {
      questionId: q.id,
      questionText: q.question_text,
      questionType: q.question_type,
      answerText: '[No answer given]'
    };
  });

  const payload = {
    session_id: sessionId,
    answers: allAnswers
  };

  try {
    setLoading(true);
    const response = await axios.post('/api/interview/submit-all', payload, {
      headers: { Authorization: `Bearer ${token}` }
    });
    setResults(response.data);
    // Clear localStorage after successful submission
    questions.forEach(q => localStorage.removeItem(`interview_q_${q.id}`));
  } catch (err) {
    console.error('Submission failed:', err);
  } finally {
    setLoading(false);
  }
};
```

6.3 Backend Implementation — Flask /submit-all Endpoint

New single-submission endpoint in Flask

```
@app.route('/api/interview/submit-all', methods=['POST'])
@jwt_required()
def submit_all_answers():
    data = request.get_json()
    session_id = data.get('session_id')
    answers = data.get('answers', []) # List of {questionId, questionText, answerText,
questionType}

    if not answers:
        return jsonify({'error': 'No answers provided'}), 400
```

```
    # Build a single prompt for holistic evaluation
    prompt = build_evaluation_prompt(answers)

    # One API call for ALL answers
    response = openai.chat.completions.create(
        model='gpt-3.5-turbo',
        messages=[
            {'role': 'system', 'content': 'You are an expert technical interviewer. Evaluate all answers
holistically.'},
            {'role': 'user', 'content': prompt}
        ],
        max_tokens=2000
    )

    result_text = response.choices[0].message.content
    parsed_results = parse_evaluation_response(result_text)

    # Save to DB
    session = InterviewSession.query.get(session_id)
    session.status = 'Completed'
    session.average_score = parsed_results['overall_score']
    db.session.commit()

    return jsonify(parsed_results)
```

Build the single holistic evaluation prompt

```
def build_evaluation_prompt(answers):
    lines = ['Evaluate this mock interview. For each answer, provide:']
    lines.append('- content_score (0-10): Accuracy and relevance of content')
    lines.append('- clarity_score (0-10): Clarity of communication')
    lines.append('- overall_score (0-10): Combined holistic score')
    lines.append('- feedback: 2-3 sentences of specific, actionable feedback')
    lines.append('- strength: One thing done well')
    lines.append('- improvement: One specific thing to improve')
    lines.append('')
    lines.append('Return your evaluation as a JSON array.')
    lines.append('')
    for i, a in enumerate(answers, 1):
        lines.append(f'Q{i} [{a["questionType"]}]: {a["questionText"]}')
        lines.append(f'Answer: {a["answerText"]}')
        lines.append('')
    return '\n'.join(lines)
```

7. FEAT-04: Downloadable PDF Report

Feature Description

After the interview evaluation results are displayed, users must be able to click a 'Download Report' button and receive a professionally formatted PDF of their entire interview session with scores and feedback.

7.1 Implementation — Frontend Only (No Backend Needed)

Use the jsPDF + html2canvas libraries to convert the results page directly into a PDF. This requires NO backend changes.

Step 1 — Install libraries

In your React project folder, run:

```
npm install jspdf html2canvas
```

Step 2 — Add Download Function to Results Component

PDF download handler

```
import jsPDF from 'jspdf';
import html2canvas from 'html2canvas';

const downloadReport = async () => {
  const reportElement = document.getElementById('interview-report');

  const canvas = await html2canvas(reportElement, {
    scale: 2,  // Higher resolution
    useCORS: true
  });

  const imgData = canvas.toDataURL('image/png');
  const pdf = new jsPDF('p', 'mm', 'a4');
  const pdfWidth = pdf.internal.pageSize.getWidth();
  const pdfHeight = (canvas.height * pdfWidth) / canvas.width;

  // Handle multi-page reports
  let heightLeft = pdfHeight;
  let position = 0;
  pdf.addImage(imgData, 'PNG', 0, position, pdfWidth, pdfHeight);
  heightLeft -= pdf.internal.pageSize.getHeight();
```

```
  while (heightLeft > 0) {
    position = heightLeft - pdfHeight;
    pdf.addPage();
    pdf.addImage(imgData, 'PNG', 0, position, pdfWidth, pdfHeight);
    heightLeft -= pdf.internal.pageSize.getHeight();
  }

  const filename = `Interview_Report_${new Date().toISOString().split('T')[0]}.pdf`;
  pdf.save(filename);
};
```

Step 3 — Wrap your Results JSX in a div with id

Wrap your results component

```
<div id='interview-report' className='report-container'>
  <h1>Interview Performance Report</h1>
  <p>Candidate: {user.name}  |  Date: {sessionDate}</p>
  <p>Overall Score: {results.overall_score}/10</p>
  {results.question_results.map((r, i) => (
    <div key={i} className='question-result'>
      <h3>Q{i+1}: {r.questionText}</h3>
      <p><strong>Your Answer:</strong> {r.answerText}</p>
      <p><strong>Content Score:</strong> {r.content_score}/10</p>
      <p><strong>Clarity Score:</strong> {r.clarity_score}/10</p>
      <p><strong>Feedback:</strong> {r.feedback}</p>
      <p><strong>Strength:</strong> {r.strength}</p>
      <p><strong>To Improve:</strong> {r.improvement}</p>
    </div>
  ))}
</div>
<button onClick={downloadReport}>Download PDF Report</button>
```

8. Bug Fix — BUG-05: STT Not Working Reliably

Bug Description

Location: Interview module — microphone recording logic

Symptom: Speech recognition either does not start, produces empty transcripts, or cuts off mid-answer.

Impact: Users cannot complete the interview at all when STT fails, making the whole interview module non-functional.

8.1 Root Cause Analysis

- The Web Speech API (window.SpeechRecognition) is not available in Firefox or Safari — only Chrome and Edge support it. The current code does not check for this.
- The recognition.continuous = false (or missing) setting causes the API to stop after the first pause, cutting off answers.
- No error handling for the 'no-speech', 'audio-capture', 'not-allowed' error codes means silent failures.
- The STT starts before the browser has confirmed microphone permission, causing the first recognition session to sometimes fail.

8.2 Complete Robust STT Solution

Add this browser check at component mount

```
useEffect(() => {
  const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
  if (!SpeechRecognition) {
    setBrowserSupported(false);
    // Show a clear message: 'Please use Google Chrome for the interview'
  }
}, []);

// Pre-request microphone permission before the interview starts
// (Put this in an 'Allow Microphone' step before Q1)
const requestMicPermission = async () => {
  try {
    await navigator.mediaDevices.getUserMedia({ audio: true });
    setMicAllowed(true);
  } catch (err) {
    setMicAllowed(false);
    alert('Microphone access denied. Please allow microphone and reload.');
  }
};
```

Handle all STT error codes

```
recognition.onerror = (event) => {
  switch(event.error) {
    case 'no-speech':
      // User was silent — this is normal, handled by silence timer
      break;
    case 'audio-capture':
      setError('No microphone found. Please connect a microphone.');
      break;
    case 'not-allowed':
      setError('Microphone permission denied. Please allow access in browser settings.');
      break;
    case 'network':
      setError('Network error. Check your internet connection.');
      break;
    default:
      console.warn('STT Error:', event.error);
  }
};

// Handle auto-stop and restart for long answers
recognition.onend = () => {
  // Web Speech API auto-stops after ~60s. Restart if still in recording phase.
  if (isRecordingRef.current) {
    recognition.start();  // Restart to continue capturing
  }
};
```

8.3 Fallback — Text Input

Always provide a text input fallback in case STT fails, so the interview can still be completed:

Always show text input as fallback

```
{!browserSupported || !micAllowed ? (
  <div>
    <p>Voice input not available. Type your answer below:</p>
    <textarea
      value={transcript}
      onChange={e => setTranscript(e.target.value)}
      rows={5} placeholder='Type your answer here...'
    />
    <button onClick={stopRecordingAndAdvance}>Next Question</button>
  </div>
```

```
) : (
  <div>
    {/* Normal voice recording UI */}
  </div>
)}
```

9. Implementation Roadmap

Use this phased plan to implement all fixes without breaking the existing working features. Each phase builds on the previous.

Phase	Tasks	Details	Est. Time
Phase 1 (Quick wins)	**Fix BUG-01, BUG-03**	Name validation (frontend + backend), Random question selection in quiz endpoint	**1-2 days**
Phase 2 (Content)	**Fix BUG-02**	Expand topic HTML content, add video URLs to database, add 40+ questions per topic	**3-5 days**
Phase 3 (STT Fix)	**Fix BUG-05**	Add browser check, mic permission flow, error handling, fallback text input	**1-2 days**
Phase 4 (Interview redesign)	**Fix BUG-04 + FEAT-01/02/03**	New interview flow: TTS + countdown + STT + localStorage save + single POST submit	**5-7 days**
Phase 5 (PDF Report)	**FEAT-04**	Install jsPDF, wrap results div, add download button	**1 day**
Phase 6 (Polish)	**Testing + UI**	End-to-end testing, UI improvements, mobile responsiveness, error message review	**2-3 days**

10. UI/UX Improvements for Industry-Level Look

Beyond fixing bugs, the following UI improvements will make the application look and feel at the level of platforms like InterviewBit or HackerRank.

10.1 Interview Page — Real Interview Feel

- Show an animated avatar or waveform animation while AI is 'speaking' (TTS playing).
- Show a pulsing red dot while the user is recording, similar to a voice recorder app.
- Show the silence countdown: '2s... 1s... submitting' before auto-advancing.
- Display the question number prominently: 'Question 3 of 8'.
- Show question type badge: 'HR Question' (blue) or 'Technical Question' (orange).

10.2 Quiz Page — Exam-like Experience

- Show a countdown timer in the top-right corner for the full quiz duration.
- Add a question navigator panel on the left showing which questions are answered/unanswered.
- Highlight selected answers with a clear visual state change (not just a radio button).
- After submission, show correct answers highlighted in green and wrong answers in red.

10.3 Dashboard — Analytics

- Use Chart.js to show a line chart of quiz scores over time (score history).
- Show a radar/spider chart of skill competency across topics (where 100% = all correct).
- Add a streak counter: 'You have practiced for 5 consecutive days!'
- Show a 'Placement Readiness Score' as a large circular progress indicator.

11. Quick Reference — All Code Locations

This table maps each fix to the exact file you need to edit in your project:

Fix	File to Edit	What to Change
Name Validation	`frontend/src/pages/Register.jsx`	Add onChange regex + error state
Name Validation (BE)	`backend/routes/auth.py (or app.py)`	Add re.compile() check before DB insert
Random Questions	`backend/routes/aptitude.py`	Add .order_by(func.random()) to query

Seen Questions	`backend/models.py`	Add UserSeenQuestion model and migration
Topic Rich Content	`backend/seed_topics.py`	Rewrite learning_html for all topics
Video in Topic Page	`frontend/src/pages/TopicDetail.jsx`	Add iframe block for video_url
STT Fix	`frontend/src/components/InterviewRecorder.jsx`	Add browser check + error handler
New Interview Flow	`frontend/src/pages/Interview.jsx`	Full redesign with TTS + STT + localStorage
New Submit Endpoint	`backend/routes/interview.py`	Add /submit-all POST endpoint
PDF Download	`frontend/src/pages/InterviewResults.jsx`	Add jsPDF + html2canvas download button

End of Document — AI Pre-Placement Trainer Bug Fix & Enhancement Plan

YSPM's Yashoda Technical Campus, Satara | AI & DS Department | 2025-26